THE CHARUTAR VIDHYA MANDAL UNIVERSITY
A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

REPORT

ON

"Restaurant Table Reservation System "

Submitted in partial fulfilment of the requirements

For the Design and Analysis of Algorithm

Submitted by

Name: Dhruv Shah

Enrollment / Roll No.: 12402080601031

Class / Semester: IT-A/5th

Guided by

Faculty Name: Prof.Minakshi.Patil

Academic Year: 2026–2027

1. ABSTRACT AND INTRODUCTION

Abstract

The Restaurant Table Reservation System is a command-line application designed to organise restaurant table bookings. It allows a user to view tables, create reservations, search for a booking, cancel a reservation, and display stored reservations. The system assigns a table that can accommodate the requested party and checks that the requested time does not overlap with an existing active reservation for the same table.

The project demonstrates core Design and Analysis of Algorithms (DAA) concepts through linear search, sorting, greedy table selection, and interval-overlap detection. It uses Java collections for temporary in-memory storage and does not require a database or graphical interface.

Introduction

Restaurant reservations require more than recording a customer's name. A booking must match the party size to a table's capacity, and the table must remain free for the entire requested time interval. If these conditions are not checked, the same table may be assigned to two customers at overlapping times.

The project models a small restaurant with a fixed set of tables. Each table has a unique ID and seating capacity. Each reservation stores a reservation ID, customer name, number of guests, table ID, date, start time, end time, and active or cancelled status.

Project overview

Item	Description
Programming language	Java 17 or later
Interface	Command-line interface (CLI)
Storage	In-memory Java collections
Algorithms	Linear search, sorting, greedy selection, interval checking
Goal	Allocate suitable tables without overlapping active bookings

Name: Dhruv Shah
Enrollment number:12402080601031

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

Problem statement

Manual reservation management can cause duplicate bookings, inefficient use of seating capacity, and difficulty finding or cancelling reservations. The system needs a method to check capacity and schedule availability before confirming a booking.

The proposed solution maintains a collection of tables and reservations. For each request, it identifies tables large enough for the party, checks their schedules, sorts valid options, and selects the smallest suitable available table. If no table qualifies, the request is rejected.

Objectives

- Develop a menu-driven restaurant reservation application in Java.
- Display restaurant tables and reservation records.
- Create bookings using party size, date, start time, and duration.
- Prevent overlapping active bookings on the same table and date.
- Search and cancel reservations using a unique reservation ID.
- Apply algorithmic methods and analyse correctness and complexity.
- Validate basic input to reduce invalid requests.

Scope and assumptions

- The demonstration restaurant has six tables with capacities 2, 2, 4, 4, 6, and 8.
- Each reservation occupies one table; combining tables is outside the scope.
- Time intervals are treated as half-open intervals [start, end).
- Cancelled reservations remain in the record but no longer block the table.
- Data is held in memory and is lost when the program exits.
- The application is a sequential, single-user educational prototype.

3. REQUIREMENTS AND SYSTEM DESIGN

Functional requirements

ID	Requirement
FR1	Display restaurant tables and capacities.
FR2	Create a reservation for a customer.
FR3	Find tables that fit the party and are free for the requested time.
FR4	Search for a reservation using its ID.
FR5	Cancel an active reservation.
FR6	Display reservations and their statuses.
FR7	List suitable tables in increasing capacity order.
FR8	Validate names, guest counts, dates, times, and duration.
FR9	Reject overlapping bookings for the same table.

Non-functional requirements

- Clarity: CLI messages should be understandable to a beginner.
- Correctness: a table must satisfy capacity and availability constraints.
- Maintainability: responsibilities are separated into classes.
- Portability: the program runs wherever a compatible Java installation is available.
- Efficiency: operations have analysable time and space requirements.

System design

The design uses three principal classes. RestaurantTable stores a table ID and capacity. Reservation stores booking details and a status that can be changed to cancelled. ReservationSystem manages tables, reservations, table selection, conflict checks, search, and cancellation. Main presents the menu and collects user input.

Tables are stored in an ArrayList. Reservations are stored in a LinkedHashMap keyed by reservation ID, preserving insertion order during iteration. The demonstration search method intentionally scans reservation values linearly so that linear search can be discussed as a DAA concept.

4. DATA STRUCTURES AND ALGORITHM OVERVIEW

Data structures

Structure	Purpose	Reason
ArrayList<RestaurantTable>	Stores tables	Simple iteration and sorting
LinkedHashMap<Integer, Reservation>	Stores reservations	Keyed storage and insertion-order iteration
RestaurantTable	Represents a table	Groups table ID and capacity
Reservation	Represents a booking	Groups customer, table, time, and status
Comparator	Orders candidates	Sorts by capacity, then table ID

Algorithm overview

The reservation workflow is a sequence of filtering, availability checking, sorting, and insertion. First, the system filters out tables whose capacity is too small. It then checks each remaining table against active reservations on the requested date. Valid tables are sorted by increasing capacity. The first table is selected and the reservation is stored.

Greedy strategy

The greedy rule is to choose the smallest currently available table that can accommodate the requested party. This reduces unused seats for the current reservation. If two tables have the same capacity, the table with the lower table ID is selected first.

This local choice is easy to explain and appropriate for a small educational system, but it does not guarantee the globally best schedule for all future bookings. A globally optimised schedule would need to consider a larger set of reservations and their interactions.

Reservation interval model

Each reservation occupies a half-open interval [start, end). A booking from 19:00 to 20:30 conflicts with a request from 20:00 to 21:00, but it does not conflict with a request beginning at 20:30. This allows one booking to begin exactly when the previous booking ends.

5. ALGORITHM DESIGN AND PSEUDOCODE

Algorithm A: Find available tables

Input: guest count g, date d, start time s, and duration m. Output: suitable available tables sorted by capacity.

```
FIND-AVAILABLE-TABLES(g, d, s, m)
candidates = empty list
e = s + m minutes
FOR each table t in tables
    IF capacity(t) >= g
        IF IS-AVAILABLE(t.id, d, s, e)
            ADD t to candidates
SORT candidates by capacity ascending, then table ID
RETURN candidates
```

Algorithm B: Check table availability

```
IS-AVAILABLE(tableID, date, requestedStart, requestedEnd)
FOR each reservation r
    IF r is active AND r.tableID = tableID AND r.date = date
        IF requestedStart < r.end AND r.start < requestedEnd
            RETURN FALSE
RETURN TRUE
```

Algorithm C: Make a reservation

```
MAKE-RESERVATION(customer, guests, date, start, duration)
candidates = FIND-AVAILABLE-TABLES(guests, date, start, duration)
IF candidates is empty: RETURN FAILURE
chosen = first candidate
create reservation with next unique ID
store reservation in memory
increment next ID
RETURN reservation
```

Algorithm D: Search and cancel

```
SEARCH(id)
FOR each reservation r
    IF r.id = id: RETURN r
RETURN NOT-FOUND

CANCEL(id)
r = SEARCH(id)
IF r missing OR already cancelled: RETURN FALSE
mark r as cancelled
RETURN TRUE
```

6. CORRECTNESS AND ALGORITHM ANALYSIS

Lemma 1: The overlap condition is correct

Two half-open intervals overlap if and only if the first interval starts before the second interval ends and the second interval starts before the first interval ends. Therefore, requestedStart < existingEnd AND existingStart < requestedEnd identifies overlapping intervals. If the intervals only touch at an endpoint, the condition is false.

Lemma 2: Every candidate table is suitable

A table is added to the candidate list only if its capacity is at least the requested guest count and its availability check succeeds. The availability check scans active reservations for the same table and date and rejects any overlap. Therefore, every candidate table is suitable and available for the requested interval.

Theorem: Sequential reservations do not double-book a table

The reservation algorithm selects a table from the candidate list. By Lemma 2, that table has enough seats and no overlapping active reservation for the requested date and time. The new reservation is then inserted. Thus, under sequential single-process execution, the new reservation cannot overlap an existing active reservation on the same table and date.

Lemma 3: The greedy choice minimises unused seats

Candidate tables are sorted by increasing capacity, with table ID used to break ties. The first candidate has the smallest capacity among all suitable available tables. Therefore, its unused capacity is minimal for this reservation.

Correctness limitation

The proof assumes requests are processed sequentially. A multi-user application would need atomic transactions or locking to prevent two simultaneous requests from both passing an availability check.

7. TIME AND SPACE COMPLEXITY

Notation

Let T be the number of restaurant tables, R the number of stored reservations, and K the number of suitable candidate tables, where K ≤ T.

Operation	Time complexity	Explanation
Display tables	O(T)	Visits every table
Search reservation	O(R)	Linear scan
Check one table	O(R)	May inspect every reservation
Find available tables	O(TR + K log K)	Scan reservations per table, then sort
Make reservation	O(TR + K log K)	Candidate search plus insertion
Cancel reservation	O(R)	Search then update status
Display reservations	O(R)	Visits every record

Derivation

The algorithm considers T tables. For each table, it may scan up to R reservations, giving O(TR) work. It then sorts K candidates, which takes O(K log K) time under comparison sorting. The total is O(TR + K log K).

Space complexity

- Stored tables and reservations: O(T + R).
- Temporary candidate list: O(T).
- Sorting auxiliary space depends on implementation and is typically proportional to K for this operation.
- Overall stored space is O(T + R), excluding temporary working space.

Possible optimisation

The current search method scans reservation values to demonstrate linear search. Since reservations are keyed by ID in a map, direct lookup by key could provide expected O(1) search time. Availability checks could also be improved by indexing reservations by table and date.

Name: Dhruv Shah
Enrollment number:12402080601031

8. TESTING AND SAMPLE RESULTS

Test cases

ID	Scenario	Expected result
TC01	Reserve for 2 guests	A suitable 2-seat table is chosen if free.
TC02	Reserve for 3 guests	A suitable 4-seat table is chosen if free.
TC03	Reserve for 5 guests	A suitable 6-seat table is chosen if free.
TC04	Reserve for 9 guests	No suitable table is available.
TC05	Overlapping request on same table	That table is rejected.
TC06	Next booking starts at prior end time	Table may be reused.
TC07	Same time on different tables	Both can succeed if suitable tables exist.
TC08	Bookings on different dates	They do not conflict.
TC09	Search existing ID	Correct details appear.
TC10	Search unknown ID	Reservation not found.
TC11	Cancel active reservation	Status becomes CANCELLED.
TC12	Cancel same reservation twice	Second cancellation fails.
TC13	Invalid guest count	Request is rejected.
TC14	Invalid date or time	User is prompted again.

Sample scenario

A customer requests a table for three guests on 1 November 2026 at 19:00 for 90 minutes. If Table 3 (capacity four) is available, it is preferred over larger tables. The booking occupies 19:00–20:30. A second request for the same table at 19:30 is rejected because the intervals overlap. A request beginning at 20:30 can reuse the table.

Testing approach

Testing includes normal cases, boundary cases, and invalid input. Capacity tests verify that a table can seat the entire party. Interval tests verify overlapping and adjacent bookings. Search and cancellation tests verify reservation identification and status changes.

9. LIMITATIONS, FUTURE ENHANCEMENTS AND CONCLUSION

Limitations

- All reservation data is lost when the program exits because storage is in memory only.
- The application does not include login, payments, email, or SMS confirmation.
- Restaurant opening hours and past-date booking restrictions are not enforced.
- Multiple tables cannot be combined for a large party.
- The program is not designed for simultaneous booking requests from multiple users.
- The greedy strategy optimises the current table choice, not the complete future schedule.

Future enhancements

- Introduce file storage or a database for persistent reservations.
- Validate opening hours and reject past dates.
- Add customer contact details and booking confirmation messages.
- Use indexes by table and date to reduce availability-checking work.
- Add transaction or locking mechanisms for concurrent requests.
- Develop a GUI or web interface while reusing the reservation logic.

Conclusion

The Restaurant Table Reservation System demonstrates the practical use of DAA concepts in a small Java application. Linear search finds reservations, sorting orders suitable tables, greedy selection chooses the smallest suitable table, and interval-overlap detection prevents conflicting active bookings.

The report describes the problem, objectives, system design, data structures, pseudocode, correctness arguments, complexity analysis, and test cases. The project is intentionally limited to a command-line interface and in-memory storage so that algorithmic design remains the main focus. It can be extended into a persistent multi-user system in future work.

References

1. Java Platform documentation: java.util collections and java.time date/time API.\n2. Standard DAA concepts: linear search, comparison sorting, greedy algorithms, asymptotic complexity, and interval overlap.\n3. Restaurant Table Reservation System project design and test scenarios prepared for this academic report.

Name: Dhruv Shah
Enrollment number:12402080601031